

# Django Interview Prep

40 Interview-Ready Questions & Model Answers  
Junior • Intermediate • Advanced • Bonus Round

---

## Contents

### Junior / Foundational Questions

1. What is Django and what is its "batteries-included" philosophy?
2. Explain the MVT (Model-View-Template) architectural pattern.
3. What is the difference between a Django project and a Django app?
4. What is the purpose of settings.py and manage.py?
5. How do you create a new Django project and an app?
6. What is the Django ORM and its advantages?
7. What is a QuerySet, and how do you filter data?
8. Explain the difference between null=True and blank=True in model fields.
9. How does URL routing work in Django?
10. What are migrations, and why are they important?

### Intermediate / Functional Questions

11. Explain the difference between function-based views (FBV) and class-based views (CBV).
12. What is middleware, and how can you create custom middleware?
13. How do you handle static files and media files in Django?
14. What are signals in Django, and when should you use them?
15. How does Django handle authentication and authorization?
16. What is CSRF protection, and how does Django implement it?
17. How do you perform form validation in Django?
18. What are the different types of model inheritance in Django?
19. How do you use the Django Admin interface and customize it?
20. What is a template context and how do you pass data to templates?

### Advanced / Senior-Level Questions

21. What is the N+1 query problem, and how do you fix it using select\_related and prefetch\_related?
22. How do you handle database transactions in Django?
23. How would you optimize slow-loading pages in a production Django app?
24. How do you manage background tasks (e.g., using Celery)?
25. What are the strategies for scaling a Django application?
26. Explain how to implement custom user models in Django.
27. How do you secure a Django application for production (e.g., security settings, environment variables)?
28. How do you handle concurrency issues (e.g., two users updating the same record simultaneously)?
29. What are the best practices for testing Django views and models?
30. Explain how to design and version a REST API using Django REST Framework (DRF).

### Additional High-Value Questions (31-40)

31. What is the difference between Serializer and ModelSerializer in DRF?
32. Explain Django's caching framework and strategies.
33. What is the difference between TemplateView, ListView, DetailView, and CreateView?
34. How does Django handle file uploads and storage backends?
35. What is the difference between get() and filter() in the Django ORM?
36. Explain Django's Meta class in models — what are common options?

37. What is the difference between `OneToOneField`, `ForeignKey`, and `ManyToManyField`?
38. How do you implement pagination in Django?
39. What is a Django management command, and how do you create a custom one?
40. Explain Django's `AppConfig` and the `ready()` method — how is it used for signal registration?

## Junior / Foundational Questions

### Q1. What is Django and what is its "batteries-included" philosophy?

Django is a high-level Python web framework that promotes rapid development and clean, pragmatic design. "Batteries-included" means it ships with most of what a typical web app needs out of the box: an ORM, an admin interface, an authentication system, a forms library, a templating engine, and security middleware. This reduces the need to hunt for and integrate third-party packages just to get a standard app running.

### Q2. Explain the MVT (Model-View-Template) architectural pattern.

MVT is Django's take on MVC. Model defines the data structure and business rules via the ORM. View contains the logic that receives a request, talks to the model, and returns a response (similar to a Controller in MVC). Template handles presentation and renders HTML. Django itself acts as the controller by routing requests through URLconf to the right view, which is why the pattern is called MVT rather than MVC.

### Q3. What is the difference between a Django project and a Django app?

A project is the entire web application: it contains global settings, the root URLconf, and WSGI/ASGI entry points. An app is a self-contained, pluggable module that handles one specific piece of functionality (e.g. blog, accounts, payments). A project is typically composed of several apps, and a well-designed app can be reused across different projects.

### Q4. What is the purpose of settings.py and manage.py?

settings.py centralizes all configuration for the project: database connection, INSTALLED\_APPS, MIDDLEWARE, template engines, static/media file paths, SECRET\_KEY, and security options. manage.py is the command-line utility for administrative tasks such as runserver, migrate, createsuperuser, startapp, and shell. It is a thin wrapper around django-admin that automatically points to the project's settings.

### Q5. How do you create a new Django project and an app?

Use the following commands, then register the new app in INSTALLED\_APPS:

```
django-admin startproject myproject
cd myproject
python manage.py startapp myapp

# settings.py
INSTALLED_APPS = [
    ...
    'myapp',
]
```

### Q6. What is the Django ORM and its advantages?

The ORM (Object-Relational Mapper) lets you interact with the database using Python classes and objects instead of raw SQL. Advantages: it is database-agnostic (switching from SQLite to PostgreSQL requires little to no code change), it reduces boilerplate, it protects against SQL injection by default, schema changes are tracked as versioned Python migrations, and the resulting code is generally more readable and maintainable than hand-written SQL.

### Q7. What is a QuerySet, and how do you filter data?

A QuerySet is a lazy, chainable representation of a database query. It is not evaluated until it is iterated, sliced, or explicitly forced (e.g. `list()`, `len()`), which lets Django optimize the final SQL. Filtering is done with field lookups:

```
Book.objects.filter(author="Rowling", published_year__gte=2000)
Book.objects.exclude(status="draft")
Book.objects.filter(title__icontains="django")
```

### Q8. Explain the difference between `null=True` and `blank=True` in model fields.

`null=True` is a database-level option: it allows the column to store NULL. `blank=True` is a validation-level option used by forms and the admin: it allows the field to be left empty in a form. A common convention is to avoid `null=True` on string-based fields (`CharField`/`TextField`) and instead rely on an empty string for 'no value', reserving `null=True` for non-string fields such as `DateField`, `IntegerField`, or `ForeignKey`.

### Q9. How does URL routing work in Django?

Django uses a `URLconf`: a list of `path()` or `re_path()` entries in `urls.py` that map URL patterns to views. The root `URLconf` is set via `settings.ROOT_URLCONF`. Apps typically define their own `urls.py` which the project includes via `include()`, keeping routing modular. Named URL patterns allow reverse lookups with `reverse()` or the `{% url %}` template tag instead of hardcoding paths.

```
# project/urls.py
urlpatterns = [
    path('admin/', admin.site.urls),
    path('blog/', include('blog.urls')),
]

# blog/urls.py
urlpatterns = [
    path('<int:pk>/', views.post_detail, name='post_detail'),
]
```

### Q10. What are migrations, and why are they important?

Migrations are Django's mechanism for propagating changes made to models into the database schema, in a version-controlled, incremental way. They are generated with `makemigrations` and applied with `migrate`. They matter because they keep the schema history auditable, allow rollback to a previous state, and keep development, staging, and production databases in sync without manual SQL scripts.

```
python manage.py makemigrations
python manage.py migrate
```

## Intermediate / Functional Questions

### Q11. Explain the difference between function-based views (FBV) and class-based views (CBV).

FBVs are plain Python functions that take a request and return a response — explicit and easy to follow for simple or highly custom logic, but harder to reuse across views. CBVs use classes (`View` and Django's generic views like `ListView`, `DetailView`, `CreateView`) which enable code reuse through inheritance and mixins, and drastically cut boilerplate for standard CRUD operations, at the cost of some 'magic' that can make control flow less obvious. FBVs suit custom logic; CBVs suit standard, repeatable patterns.

## Q12. What is middleware, and how can you create custom middleware?

Middleware is a chain of hooks that process every request on the way in and every response on the way out, in the order defined by the MIDDLEWARE setting. Built-in examples handle sessions, authentication, and CSRF. Custom middleware is a callable class:

```
class TimingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        start = time.time()
        response = self.get_response(request)
        response['X-Response-Time'] = str(time.time() - start)
        return response
```

## Q13. How do you handle static files and media files in Django?

Static files (CSS, JS, images shipped with the code) are configured via STATIC\_URL and STATICFILES\_DIRS, and collected into STATIC\_ROOT with collectstatic for production, where they are served by a web server or CDN rather than Django itself. Media files (user uploads) are configured via MEDIA\_URL and MEDIA\_ROOT and handled through FileField/ImageField; in production they are typically stored on cloud storage such as S3 rather than local disk.

## Q14. What are signals in Django, and when should you use them?

Signals let decoupled parts of an application react when something happens elsewhere, e.g. post\_save, pre\_delete, or m2m\_changed. A common use is automatically creating a Profile when a User is created. They are best used sparingly: overusing signals hides side effects and makes code flow hard to trace, so an explicit function call is often preferable when the caller is known.

```
from django.db.models.signals import post_save
from django.dispatch import receiver

@receiver(post_save, sender=User)
def create_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.create(user=instance)
```

## Q15. How does Django handle authentication and authorization?

django.contrib.auth provides a User model, password hashing, and login/logout views. Authentication verifies who the user is (session-based login by default). Authorization checks what they're allowed to do, via is\_staff/is\_superuser flags, the permissions and groups system, decorators like @login\_required and @permission\_required, or PermissionsMixin on CBVs. For APIs, token or JWT authentication is commonly added through Django REST Framework.

## Q16. What is CSRF protection, and how does Django implement it?

Cross-Site Request Forgery protection stops a malicious site from tricking an authenticated user's browser into submitting an unwanted request. Django implements it via CsrfViewMiddleware together with the {% csrf\_token %} template tag, which embeds a hidden token that must be echoed back on POST requests. AJAX or API clients must send the token in an X-CSRFToken header.

## Q17. How do you perform form validation in Django?

Forms and ModelForms declare fields with built-in validators. Calling `is_valid()` runs field-level `clean_()` methods, then the form-level `clean()` method for cross-field checks, and populates `form.errors` on failure.

```
class SignupForm(forms.Form):
    age = forms.IntegerField()

    def clean_age(self):
        age = self.cleaned_data['age']
        if age < 18:
            raise forms.ValidationError('Must be 18 or older')
        return age
```

## Q18. What are the different types of model inheritance in Django?

Abstract base classes (`Meta.abstract = True`) share fields/methods across models without creating a table for the base itself. Multi-table inheritance gives each model, including the base, its own table, linked by an implicit `OneToOneField` — used when the parent should also exist as a queryable object in its own right. Proxy models reuse the same table but change Python-level behavior (e.g. default manager, ordering) without adding new fields.

## Q19. How do you use the Django Admin interface and customize it?

The admin app (`django.contrib.admin`) auto-generates a management UI from registered models. Customize it via a `ModelAdmin` subclass controlling `list_display`, `list_filter`, `search_fields`, `readonly_fields`, `inlines` for related models, and custom actions.

```
@admin.register(Book)
class BookAdmin(admin.ModelAdmin):
    list_display = ('title', 'author', 'published_year')
    list_filter = ('author',)
    search_fields = ('title',)
```

## Q20. What is a template context and how do you pass data to templates?

The context is a dictionary of variables made available inside a template during rendering, passed as the third argument to `render()`. Context processors (configured in `TEMPLATES`) automatically inject common variables, such as `request` or `user`, into every template's context without each view needing to add them manually.

```
def book_list(request):
    books = Book.objects.all()
    return render(request, 'books/list.html', {'books': books})
```

# Advanced / Senior-Level Questions

## Q21. What is the N+1 query problem, and how do you fix it using `select_related` and `prefetch_related`?

It happens when iterating over a queryset triggers one extra database query per row to fetch a related object (1 query for the list + N queries for related fields). `select_related` performs a SQL JOIN and works for `ForeignKey`/`OneToOne` relations, fetching everything in a single query. `prefetch_related` runs a separate query per relation and joins the results in Python, which is needed for `ManyToMany` and reverse `ForeignKey` relations.

```
# N+1 problem
for book in Book.objects.all():
    print(book.author.name)    # 1 query per book
```

```
# Fixed
for book in Book.objects.select_related('author'):
    print(book.author.name)    # 1 query total
```

## Q22. How do you handle database transactions in Django?

Wrap related operations in `transaction.atomic()`, used as a decorator or context manager, so they either all succeed or all roll back on an exception — essential for operations like transferring funds between two accounts.

```
from django.db import transaction

with transaction.atomic():
    account1.balance -= 100
    account1.save()
    account2.balance += 100
    account2.save()
```

## Q23. How would you optimize slow-loading pages in a production Django app?

Typical steps: eliminate N+1 queries with `select_related/prefetch_related`; add database indexes on frequently filtered/ordered columns; profile with `django-debug-toolbar` or `django-silk`; cache expensive views or queriesets with `Redis/Memcached`; paginate large result sets; trim fetched columns with `only()/defer()`; move slow operations to Celery background tasks; serve static/media via a CDN; and enable gzip/HTTP caching headers.

## Q24. How do you manage background tasks (e.g., using Celery)?

Celery is a distributed task queue that integrates with Django to run work asynchronously, backed by a broker such as `Redis` or `RabbitMQ`. Tasks are defined with `@shared_task` and triggered with `.delay()` or `.apply_async()` so the web request isn't blocked. Celery Beat schedules recurring/periodic tasks such as nightly reports.

```
from celery import shared_task

@shared_task
def send_welcome_email(user_id):
    user = User.objects.get(id=user_id)
    send_mail(...)

# calling it
send_welcome_email.delay(user.id)
```

## Q25. What are the strategies for scaling a Django application?

Run stateless app servers horizontally behind a load balancer; add database read replicas for read-heavy workloads; introduce a caching layer (`Redis/Memcached`); offload static/media delivery to a CDN; push heavy or slow work to async task queues; use connection pooling (e.g. `PgBouncer`); and, at very large scale, consider database sharding/partitioning or moving I/O-bound views to async views under ASGI.

## Q26. Explain how to implement custom user models in Django.

Best practice is to define a custom user model at the very start of a project, even if it initially matches the default, since swapping the user model later is painful. Extend `AbstractUser` to keep the default fields and add extras, or `AbstractBaseUser` for full control over fields and authentication. Set `AUTH_USER_MODEL` before running the first migration.

```
# settings.py
AUTH_USER_MODEL = 'accounts.CustomUser'

# models.py
```

```
class CustomUser(AbstractUser):
    phone_number = models.CharField(max_length=20, blank=True)
```

## Q27. How do you secure a Django application for production (e.g., security settings, environment variables)?

Set `DEBUG = False` and a correct `ALLOWED_HOSTS`; load `SECRET_KEY` and credentials from environment variables, never hardcoded; enforce HTTPS via `SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`, and `SECURE_HSTS_SECONDS`; set X-Frame-Options; keep dependencies patched; restrict or relocate the admin URL; and run `python manage.py check --deploy` before release to catch missing hardening settings.

## Q28. How do you handle concurrency issues (e.g., two users updating the same record simultaneously)?

Pessimistic locking: `select_for_update()` inside `transaction.atomic()` locks the row until the transaction completes, so a second writer waits. Optimistic locking: add a version field and only commit the update if the version hasn't changed since it was read, otherwise treat it as a conflict. For simple counters, use `F()` expressions so the increment happens atomically in the database rather than via a Python read-modify-write cycle.

```
from django.db.models import F

Product.objects.filter(id=1).update(stock=F('stock') - 1) # atomic, avoids race condition
```

## Q29. What are the best practices for testing Django views and models?

Use Django's `TestCase`, which wraps each test in a database transaction that is rolled back afterward for isolation and speed. Separate fast unit tests (models, forms, utility functions) from integration tests that exercise views via the test Client. Use factories (e.g. Factory Boy) instead of large fixture files, mock external services, cover both happy paths and edge cases such as permission failures, and track coverage with `coverage.py` inside CI.

## Q30. Explain how to design and version a REST API using Django REST Framework (DRF).

Use `ModelSerializer` for standard CRUD serialization/validation, `ViewSet`s combined with `Routers` to generate conventional REST endpoints, and `permission_classes/authentication_classes` to control access. Paginate list endpoints. Common versioning strategies are URL path versioning (`/api/v1/...`), a query parameter, a custom Accept header, or namespace versioning; URL path versioning is usually preferred for its simplicity and visibility.

```
class BookViewSet(viewsets.ModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
    permission_classes = [IsAuthenticatedOrReadOnly]

# urls.py
router.register('books', BookViewSet)
urlpatterns = [path('api/v1/', include(router.urls))]
```

## Additional High-Value Questions (31-40)

### Q31. What is the difference between `Serializer` and `ModelSerializer` in DRF?

A plain Serializer requires you to explicitly declare every field and write create()/update() yourself, similar to a Django Form — full control, more boilerplate. ModelSerializer auto-generates fields from a model and provides default create()/update() implementations, which is faster to write and keeps serializers in sync with model changes, at the cost of a little 'magic' that you can still override where needed.

### Q32. Explain Django's caching framework and strategies.

Django's cache framework supports pluggable backends: local-memory, filesystem, Memcached, and Redis (via django-redis). It offers several levels of granularity: per-site caching via middleware, per-view caching with the @cache\_page decorator, template-fragment caching with the {% cache %} tag, and a low-level API (cache.get/cache.set) for caching arbitrary data such as expensive queryset results. The right level depends on how often the underlying data changes.

```
from django.views.decorators.cache import cache_page

@cache_page(60 * 15) # cache this view for 15 minutes
def expensive_view(request):
    ...
```

### Q33. What is the difference between TemplateView, ListView, DetailView, and CreateView?

TemplateView simply renders a template with an optional static context — no model logic. ListView displays a queryset of objects, with built-in pagination support. DetailView displays a single object looked up by pk or slug. CreateView/UpdateView/DeleteView wrap a ModelForm to handle the full create/edit/delete workflow, including validation and redirect-on-success, with minimal code.

### Q34. How does Django handle file uploads and storage backends?

FileField and ImageField store only the file's path/name in the database; the actual bytes are handled by a storage backend, defaulting to FileSystemStorage under MEDIA\_ROOT. In production, the default is usually swapped for a cloud backend (e.g. django-storages with Amazon S3 or Google Cloud Storage) via DEFAULT\_FILE\_STORAGE, so uploads survive server restarts/redeploys and scale across multiple app instances. Always validate file size and type before saving.

### Q35. What is the difference between get() and filter() in the Django ORM?

get() returns a single model instance and raises Model.DoesNotExist if nothing matches, or Model.MultipleObjectsReturned if more than one row matches — use it only when exactly one result is expected (e.g. by primary key). filter() always returns a QuerySet, which may be empty, and never raises for zero or multiple matches, making it the safer default when the result count isn't guaranteed.

```
try:
    book = Book.objects.get(isbn=isbn)
except Book.DoesNotExist:
    book = None

books = Book.objects.filter(genre='sci-fi') # always a QuerySet, may be empty
```

### Q36. Explain Django's Meta class in models — what are common options?

The inner Meta class configures model-level behavior without adding fields. Common options: ordering (default query order), unique\_together or UniqueConstraint (composite uniqueness), db\_table (override the generated table name), verbose\_name/verbose\_name\_plural (admin display names), indexes (extra database indexes),

abstract (make it a non-table base class), and constraints (e.g. CheckConstraint for validation at the database level).

```
class Meta:
    ordering = ['-created_at']
    constraints = [
        models.UniqueConstraint(fields=['user', 'course'], name='unique_enrollment')
    ]
```

### Q37. What is the difference between OneToOneField, ForeignKey, and ManyToManyField?

ForeignKey models a many-to-one relationship, e.g. many books can belong to one author. OneToOneField enforces exactly one match on each side, e.g. a User linked to exactly one Profile. ManyToManyField models a many-to-many relationship, e.g. students enrolled in multiple courses and courses having multiple students; Django creates a hidden join table automatically, or you can specify a custom `through` model when the relationship itself needs extra fields (such as an enrollment date).

### Q38. How do you implement pagination in Django?

For server-rendered pages, use Django's built-in Paginator/Page classes directly, or simply set paginate\_by on a ListView. For APIs, DRF provides PageNumberPagination and LimitOffsetPagination for typical use cases, and CursorPagination for large or frequently changing datasets, since it avoids the 'page drift' that happens when rows are inserted or deleted between page requests.

```
class BookListView(ListView):
    model = Book
    paginate_by = 20  # 20 books per page
```

### Q39. What is a Django management command, and how do you create a custom one?

A management command is a custom CLI script invoked through manage.py, useful for cron jobs, data imports/exports, and one-off maintenance scripts. Create it under /management/commands/.py, subclass BaseCommand, implement handle(), and optionally add\_arguments() for CLI options.

```
# myapp/management/commands/cleanup_old_sessions.py
from django.core.management.base import BaseCommand

class Command(BaseCommand):
    help = 'Deletes expired sessions'

    def handle(self, *args, **options):
        count, _ = Session.objects.filter(expire_date__lt=now()).delete()
        self.stdout.write(f'Deleted {count} sessions')
```

### Q40. Explain Django's AppConfig and the ready() method — how is it used for signal registration?

AppConfig (defined in apps.py) holds metadata about an app, such as its name, label, and default\_auto\_field. Its ready() method runs exactly once, after the entire app registry has finished loading. This is the correct place to import and connect signal handlers, because importing them directly inside models.py can trigger circular imports or double-registration; ready() guarantees all apps and models are already available.

```
# apps.py
class MyAppConfig(AppConfig):
    name = 'myapp'
```

```
def ready(self):  
    import myapp.signals # connects @receiver handlers
```